

GraphRAG 入门教程：从原理到实战 – 指南

GraphRAG 入门教程：从原理到实战



1. 什么是 GraphRAG?

GraphRAG 是一种结构化的、分层的检索增强生成 (Retrieval-Augmented Generation, 简称 RAG) 方法

- 和传统的 RAG 不同, GraphRAG 不仅仅依赖文本相似度搜索, 而是先把文本转成 **知识图谱 (Knowledge Graph)**, 再基于图谱结构来检索和生成答案。
- 【回答质量高, 但 token 消耗大、生成时间久, 所以使用代价较高, 个人部署不建议 ~~~】

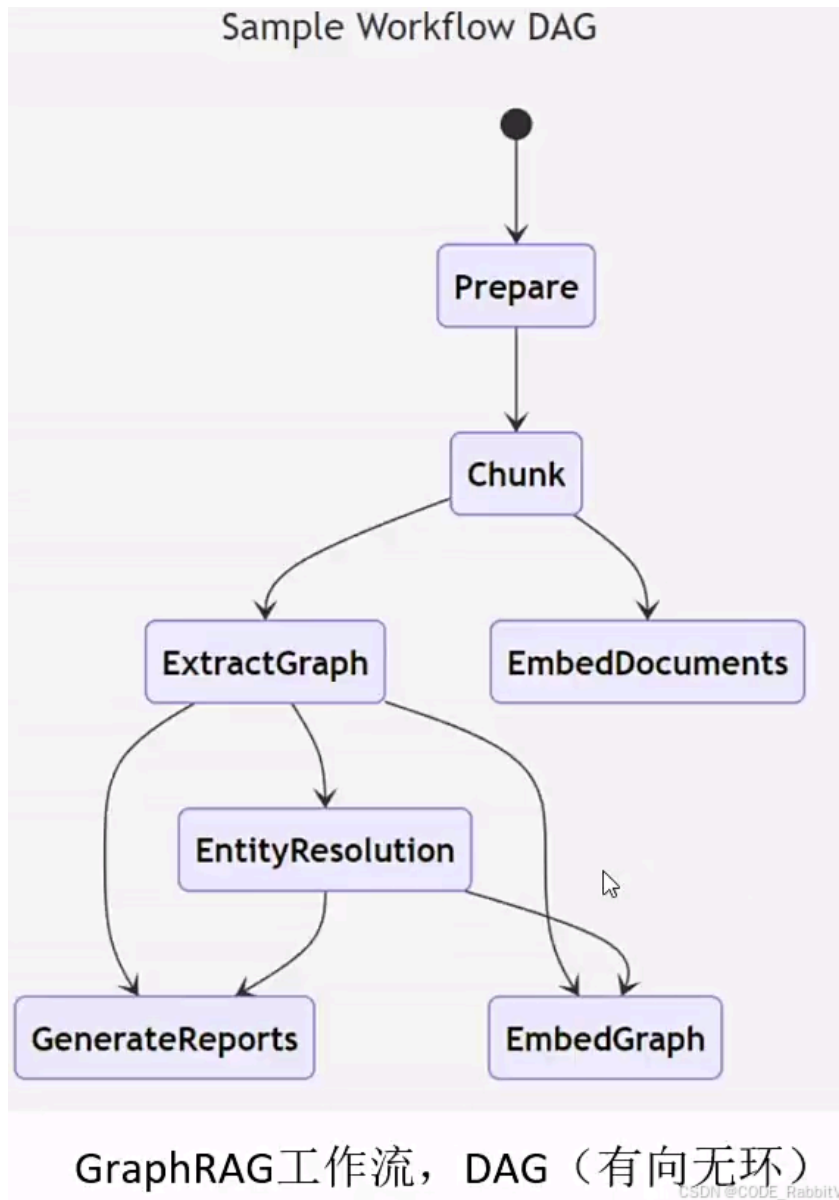
简单来说:

- 传统 RAG: 找到和问题最像的文本片段 → 直接生成答案。
- GraphRAG: 先提取实体和关系 → 构建知识图谱 → 检索更精准的信息 → 生成更丰富的答案。

这样做的好处是:

- 更好地处理跨文档、多跳推理的问题。
- 能发现信息之间的隐含联系, 而不仅是关键词匹配。

2. GraphRAG 原理



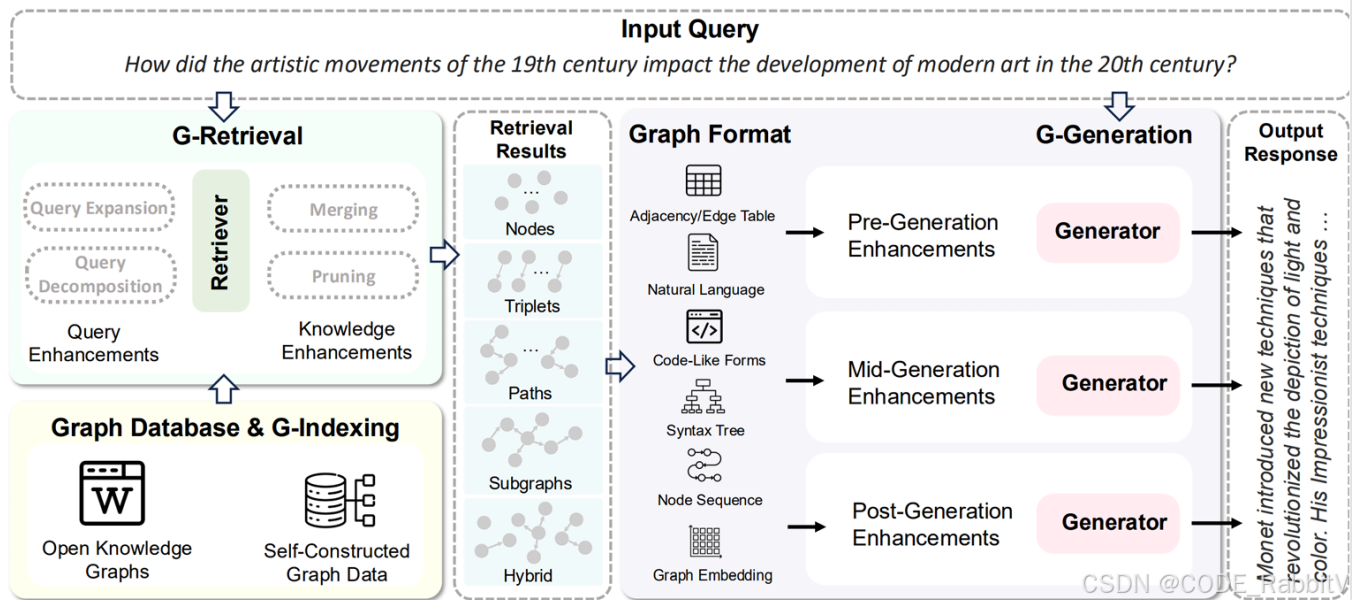
GraphRAG 核心思想：

1. 从原始文本中提取知识图谱
 - 节点（实体）+ 边（关系）
2. 构建社区层级（Community Levels）
 - 发现信息的群体结构，比如哪些实体属于同一主题、组织或地理位置。
3. 为这些社区生成摘要
 - 让模型理解某个社区的整体背景。
4. 结合 RAG 任务执行问答
 - 通过图谱检索找到最相关的信息，再生成答案。

类比一下：传统 RAG 是“在书里找几段相关的句子”，GraphRAG 是“先画一张信息关系图，再从图上找到最优路径来回答问题”。

3. GraphRAG 的流程

GraphRAG 的整体流程可以分为两大部分：索引阶段 和 查询阶段。



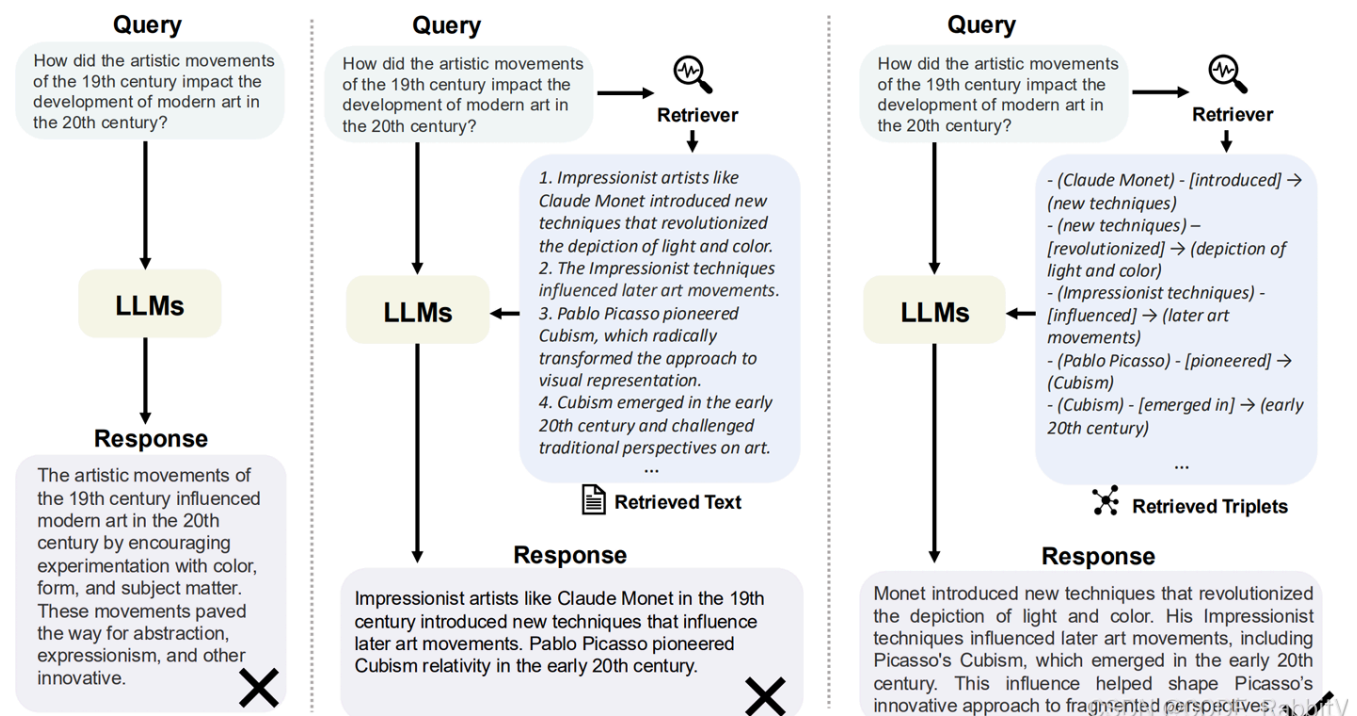
3.1 索引阶段 (Indexing)

1. 切分文本: 把大文本库切成可处理的小单元 (TextUnits)。
2. 提取实体、关系和关键声明: 用 LLM 从文本中抽取人物、地点、事件等实体及它们之间的关系。
3. 构建知识图谱: 节点是实体, 边是关系。
4. 社区检测与聚类: 用 Leiden 等算法将图谱分成若干社区。
5. 生成社区摘要: 用 LLM 总结社区关键信息。
6. 存储到图数据库: 方便后续高效检索。

3.2 查询阶段 (Querying)

1. 解析用户问题: 分解查询、识别涉及的实体。
2. 全局检索: 从社区摘要中获取整体背景。
3. 局部检索: 深入邻居节点和相关关系, 获取细节。
4. 生成答案: 将检索结果交给 LLM, 生成自然语言回答。

4. 示例对比



假设问题是:

“Query: 19世纪的艺术运动是如何影响20世纪现代艺术的发展的？”

• 传统 LLM 直接回答：

“19 世纪的艺术运动通过鼓励对色彩、形式和主题的实验，影响了 20 世纪的现代艺术……”

笼统，没有细节链路！

• 普通 RAG 回答（检索文本片段）：

检索：1. 像克劳德·莫奈这样的印象派艺术家引入了新技术，彻底改变了对光和颜色的描绘。2. 印象派的技法影响了后来的 ...

回答：莫奈引入新技术，改变了光和色彩的描绘；印象派影响了后来的艺术运动；毕加索开创立体主义……

回答依旧可能割裂！

• GraphRAG 回答（基于知识图谱）：

检索：(莫奈) – [引进] → (新技术) – [革新] → (光和颜色的描绘) ...

回答：“莫奈引入的新技术彻底改变了光和色彩的描绘，他的印象派技巧影响了后来的艺术运动，包括 20 世纪初出现的毕加索立体主义。这种影响帮助塑造了毕加索对碎片化视角的创新方法。”

优势：

- 有因果链条（莫奈 → 新技术 → 印象派 → 立体主义）
- 信息更连贯

5. 实战：如何跑 GraphRAG

官方代码地址：<https://github.com/microsoft/graphrag>

Step 1. 克隆代码

```
1 git clone https://github.com/microsoft/graphrag.git
2 cd graphrag
```

Step 2. 安装依赖、初始化 & 配置

```
1 pip install poetry
2 poetry install
3 poetry run poe index --init--root .
```

- 正确运行后，此处会在 graphrag 目录下生成 output、prompts、.env、settings.yaml 文件
- 之后需要对 .env 文件配置 GRAPHRAG_API_KEY、修改 settings.yaml 设置 model

Step 3. 准备数据，放到 ./input 目录下

- 准备一个包含多篇文档的文本数据集（比如企业内部报告）
- 格式可以是 .txt / .csv / .json

Step 4. 构建索引（文本越长越久）

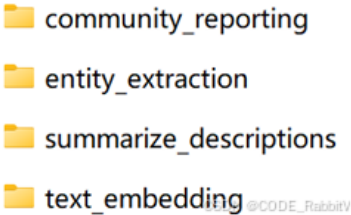
```
1 python -m graphrag.index --init
```

之后会看到类似如下的一些生成信息：

```
1 create_base_text_units
2 ...
3 create_base_extracted_entities
4 ...
5 create_summarized_entities
6 ...
7 create_base_entity_graph
8 ...
9 create_final_entity
```

```
10 ...
11 create_final_communities
12 ...
13 ...
14 ...
15 All workflows completed successfully.
```

- 正确运行后，会在 ./cache 文件夹下面生成4个文件夹，方便后续进行提问



Step 5. 运行查询

```
1 python -m graphrag.query--root ./cases --method global "你的问题" ## global 模式
```

- **local** 答案生成（代价高）：针对具体问题，GraphRAG通过结合元素和元素摘要生成初步答案，这些答案来源于GraphRAG中的特定社区；
- **global** 答案生成（代价非常高）：对于需要涵盖整个数据集的全局性问题，GraphRAG采用Map-Reduce机制，将所有社区的初步答案组合起来。

6. 适用场景

GraphRAG 特别适合：

- **跨文档问答**：多个文档中信息关联的问题。
- **多跳推理**：需要从多个实体关系链推理出答案。
- **知识管理**：企业内部知识库、科研资料等。
- **长文本总结**：社区层级摘要可以提炼核心脉络。

7. 总结

GraphRAG 让 RAG 不再是“找最像的文本”，而是“基于关系图谱推理回答”。

它的关键价值在于：

- 结构化信息 → 让知识更可检索、可推理。
- 分层摘要 → 快速获得全局视野与细节。
- 更适合复杂、跨域的问题。

如果你平时做的问答任务经常遇到：

- 回答内容碎片化、不连贯
- 模型找不到跨文档的关键信息

那 GraphRAG 值得你尝试。

posted on 2025-08-11 08:00 [ljbguanli](#) 阅读(1454) 评论(0) [收藏](#) [举报](#)